

Updating a state (continue)

- We can do a sequence of updates on a state, such as $\sigma[x \mapsto 0][y \mapsto 8]$. Here, we update the state left-to-right.
 - For example, let $\sigma = \{x = 2, y = 6\}$, then $\sigma[x \mapsto 0][y \mapsto 8] = \{x = 0, y = 6\}[y \mapsto 8] = \{x = 0, y = 8\}$.

1. True or False

- a. Let variables $x \neq y$, then $\sigma[x \mapsto 0][y \mapsto 8] = \sigma[y \mapsto 8][x \mapsto 0]$

True. x will be bound with 0 and y will be bound with 8.

- b. Let variables $x \neq y$, then $\sigma[x \mapsto 0][y \mapsto 8] \equiv \sigma[x \mapsto 0][y \mapsto 8]$

True, they are exactly the same text.

- c. Let variables $x \neq y$, then $\sigma[x \mapsto 0][y \mapsto 8] \equiv \sigma[y \mapsto 8][x \mapsto 0]$

False. Updating procedures are different.

- d. $\sigma[x \mapsto 0][x \mapsto 8] = \sigma[x \mapsto 8]$

True. x will be bound with 8.

- e. $\sigma[x \mapsto 0][x \mapsto 8] \equiv \sigma[x \mapsto 8]$

False. Updating procedures are different.

2. Let $\sigma = \{x = 1, z = 2\}$, find state $\sigma[x \mapsto 2][z \mapsto \sigma[z \mapsto 3](x + z) + 10]$.

$$\begin{aligned} \sigma[x \mapsto 2][z \mapsto \sigma[z \mapsto 3](x + z) + 10] &= \{x = 2, z = 2\}[z \mapsto \sigma[z \mapsto 3](x + z) + 10] \\ &= \{x = 2, z = 2\}[z \mapsto \{x = 1, z = 3\}(x + z) + 10] \\ &= \{x = 2, z = 2\}[z \mapsto 4 + 10] \\ &= \{x = 2, z = 2\}[z \mapsto 14] \\ &= \{x = 2, z = 14\} \end{aligned}$$

- How to understand updating values in an array? What do we do if we want to update the value in $b[0]$? Since we handle array as a function from an index to the value stored, here let's expand the notion of updating states to updating functions.
- If δ is a function and α and β are valid elements of the domain and range of δ respectively, then the update of δ at α with β , written $\delta[\alpha \mapsto \beta]$, is the function defined by $\delta[\alpha \mapsto \beta](\gamma) = \beta$ if $\gamma = \alpha$ and $\delta[\alpha \mapsto \beta](\gamma) = \delta(\gamma)$ if $\gamma \neq \alpha$.
 - Note that, if we consider state as a function, then the definition of updating a state follows the above definition as well. The only difference is that the α and γ here are values.
For example, let function $\delta = \{(4, 6), (3, 7), (2, 5)\}$, then $\delta[2 \mapsto 3] = \{(4, 6), (3, 7), (2, 3)\}$, because $\delta(2)$ is updated from 5 to 3. We also can get $\delta[2 \mapsto 3](2) = 3$, $\delta[2 \mapsto 3](3) = 7$.
- Say σ is a (proper) state with an array b , with $\eta =$ the function $\sigma(b)$. If α is a valid index value for b , then $\sigma[b[\alpha] \mapsto \beta]$ means $\sigma[b \mapsto \eta[\alpha \mapsto \beta]]$. So, updating σ at $b[\alpha]$ with β involves updating σ with an updated version of η , namely $\eta[\alpha \mapsto \beta]$, as the value of b .
 - For example, $\sigma = \{x = 3, b = (2, 4, 6)\}$, then $\sigma[b[0] \mapsto 8] = \{x = 3, b = (8, 4, 6)\}$. Here, $\sigma(b) = (2, 4, 6)$, which can also be written as $\{(0, 2), (1, 4), (2, 6)\}$, so $\sigma(b)[0 \mapsto 8] = \{(0, 8), (1, 4), (2, 6)\} = (8, 4, 6)$.

Satisfaction of Quantified Predicates

- If a variable is bounded in a quantifier, we consider it is defined “locally” and only being used inside of this quantified predicate.
3. True or False?
- a. $(\forall y \in \mathbb{Z}. y > 1 \rightarrow y > 0) \equiv (\forall x \in \mathbb{Z}. x > 1 \rightarrow x > 0)$
True. For a quantified predicate, which local variable we use does not change the way this text is processed.
 - b. $\{x = 1\}$ is proper for predicate $\forall y \in \mathbb{Z}. y > 1 \rightarrow y > 0$.
True. Variable y is defined inside of the universally quantified predicate, to evaluate this universal we don't need y to be defined in a state.
 - c. $\{x = 1\}$ is proper for predicate $(\forall y \in \mathbb{Z}. y > 1 \rightarrow y > 0) \wedge y < 5$
False. There is a y appears outside of the quantified predicate (the $y < 5$), we need y being defined in a state to evaluate.
- $\sigma \models \exists x \in S. p$ if for one or more **witness values** $\alpha \in S$, it's the case that $\sigma[x \mapsto \alpha] \models p$.
4. True or False?
- a. $\{z = 4, x = -5\} \models \exists x. x \geq z$?
True. We can find $x = 5$ such that $\{z = 4, x = -5\}[x \mapsto 5] = \{z = 4, x = 5\}$ satisfies $x \geq z$.
 - b. $\sigma \models \exists x. x^2 \leq 0$?
True. x has domain $\mathbb{Z}$, and we can find $x = 0$ such that $\sigma[x \mapsto 0]$ satisfies $x^2 \leq 0$.
5. Which of the following state satisfies $x < 3 \wedge \exists x. b[x] > 5$?
- a. $\{x = 0, b = (2, 4, 3, 1)\}$
 - b. $\{x = 1, b = (1, 3, 5, 7)\}$
 - c. $\{x = 2, b = (1, 3, 5, 4)\}$
 - d. $\{x = 3, b = (6, 5, 3, 1)\}$
- $\sigma \models \forall x \in S. p$ if for every value $\alpha \in S$, we have $\sigma[x \mapsto \alpha] \models p$.
6. True or False.
- a. $\{y = 1\} \models \forall x \in \mathbb{Z}. x^2 \geq y - 1$?
True. $\{y = 1\}(y - 1) = 0$, and we know that for all integer α , we have $\alpha^2 \geq 0$.
 - b. $\{x = -1\} \models \forall x \in \mathbb{Z}. x^2 \geq x$?
True. We know that for all integer α , we have $\alpha^2 \geq \alpha$.
- How to understand “a state doesn't satisfy a quantified predicate”? Without consider the possible runtime error during the evaluation, we can use “ $\sigma \not\models p \Leftrightarrow \sigma \models \neg p$ ” then apply DeMorgan's Law here:
 - $\sigma \not\models \exists x \in S. p \Leftrightarrow \sigma \models \neg \exists x \in S. p \Leftrightarrow \sigma \models \forall x \in S. \neg p$
 - $\sigma \not\models \forall x \in S. p \Leftrightarrow \sigma \models \neg \forall x \in S. p \Leftrightarrow \sigma \models \exists x \in S. \neg p$

Validity of Predicates

- Let p be a proposition or predicate. $\models p$ means $\sigma \models p$ for all proper σ , and we say p is **valid**. In other words, we can say “It is always true that, p ”.
 - $(\models p) \Leftrightarrow (\forall \sigma \in S. \sigma \models p)$ (where S is the collection of all well-formed states that are proper for p)
- $\not\models p$ means $\sigma \not\models p$ for some proper σ , and we say p is **invalid**. In other words, we can say “ p is not always true”.

- $(\not\models p) \Leftrightarrow (\exists \sigma \in S. \sigma \not\models p)$ (where S is the collection of all well-formed states are proper for p)

7. True or False.

a. $\models x > 1 \rightarrow x^2 > x$

True, because all states satisfy $x > 1 \rightarrow x^2 > x$

b. $\models x > 1 \wedge x^2 > x$

False, because $\{x = 0\} \not\models x > 1 \wedge x^2 \geq x$

c. $\models \forall x. x > 1 \rightarrow x^2 > x$

True

We need to check that whether $\forall \sigma. \sigma \models \forall x. x > 1 \rightarrow x^2 > x$. While checking whether some $\sigma \models \forall x. x > 1 \rightarrow x^2 > x$, we don't need any bindings in σ to decide satisfaction, since this universal quantified x and its body contains only variable x as well. Therefore, we only need to check whether any one $\sigma \models \forall x. x > 1 \rightarrow x^2 > x$, and we can simple pick $\sigma = \emptyset$ to process.

d. $\models \exists x. x > 1 \wedge x^2 > x$

True

- If p is a predicate about x and x is not a variable being bounded in p , then $(\models p) \Leftrightarrow (\models \forall x. p)$.

8. Is the following predicate valid?

$$\exists y. y \neq 0 \wedge x * y \neq 0$$

$\exists y. y \neq 0 \wedge x * y \neq 0$ is a predicate about x and x is not the variable being bounded here, so semantically its validity is equivalent to $\models \forall x. \exists y. y \neq 0 \wedge x * y \neq 0$; then it is easier to see this predicate is invalid.

To show it is invalid, we can argue that:

$$\not\models \exists y. y \neq 0 \wedge x * y \neq 0$$

$$\Leftrightarrow \exists \sigma. \sigma \not\models \exists y. y \neq 0 \wedge x * y \neq 0$$

definition of invalid

$$\Leftrightarrow \exists \sigma. \sigma \models \neg \exists y. y \neq 0 \wedge x * y \neq 0$$

definition of $\sigma \not\models p$

$$\Leftrightarrow \exists \sigma. \sigma \models \forall y. \neg(y \neq 0 \wedge x * y \neq 0)$$

DeMorgan's Law

$$\Leftrightarrow \exists \sigma. \sigma \models \forall y. y = 0 \vee x * y = 0$$

DeMorgan's Law

We can find that $\sigma = \{x = 0\}$ is a witness, since for all possible values of y , we always have $y = 0$ or $x * y = 0$.

Syntax of Statements in Our Programming Language

- **Program:** A program is simply a statement, typically a sequence statement.
- In general, a statement is a standalone unit of execution whose purpose is not creating a value (opposite to expression) but creating changes in memory. We usually use letter S to represent a statement in our programming language.
- We introduce 5 types of statements here, and we will introduce more in future classes.
 - **No-op** statement: **skip**
It simply means "do nothing".
 - **Assignment** statement: $v := e$ or $b[e_0][e_1] \dots [e_{n-1}] := e$
Assigning expression e to variable v or assigning expression e to a certain index in an n -dimensional array b .

- **Sequence** statement: $S; S'$
Do S then do S' . Note that S' can be another sequence statement, then we have a longer sequence like: $S_1; S_2; S_3$.
- **Conditional** statement: **if B then S_1 else S_2 fi**
Do S_1 if B is evaluated to *True*, do S_2 if B is evaluated to *False*.
 - A conditional statement and a conditional expression can look alike; we tell one another by context. Note that S_1 and S_2 both must be statements.
 - When S_2 is a no-op statement, then we can simplify it from **if B then S_1 else skip fi** to **if B then S_1 fi** so we don't need to formally define a **if – then** statement.
- **Iterative** statement: **while B do S od**
A “while loop” with loop condition B and do S in each iteration.
 - We don't have “**for** loops” in our language but we can simulate it using **while – do**. For example, if we need something like “**for $x = e_1$ to e_2 do S ”**, we can give

$$x := e_1; \text{while } x \leq e_2 \text{ do } S; x := x + 1 \text{ od}$$
- We use “ E ” to represent an empty statement. Note that the statement **skip** is still one operation, but E means there are no operations.